

MODEL COMPRESSION

Quantization & Pruning in LLMs

*Efficient compression techniques to reduce model size,
memory footprint, and inference latency*

The Scale Problem in LLMs

Why Compression Matters

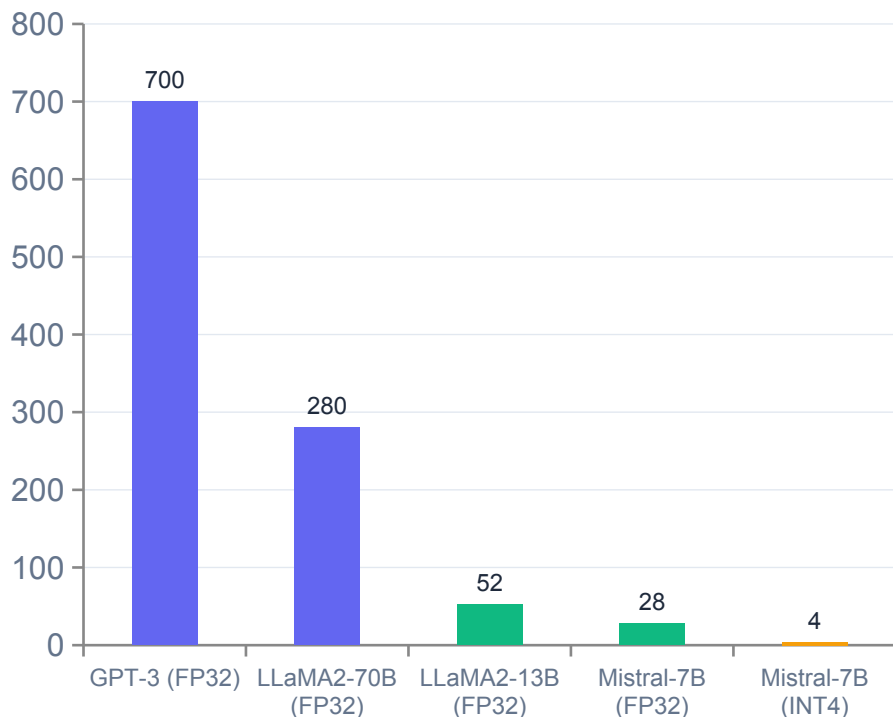
- Modern LLMs require hundreds of GB of VRAM at full FP32 precision
- Most consumer GPUs have only 8 - 24 GB of VRAM
- Inference costs at scale are prohibitively expensive
- Mobile and edge deployment is impossible without compression



Compression = Accessibility

Quantization & pruning unlock LLM deployment on consumer hardware and at the edge.

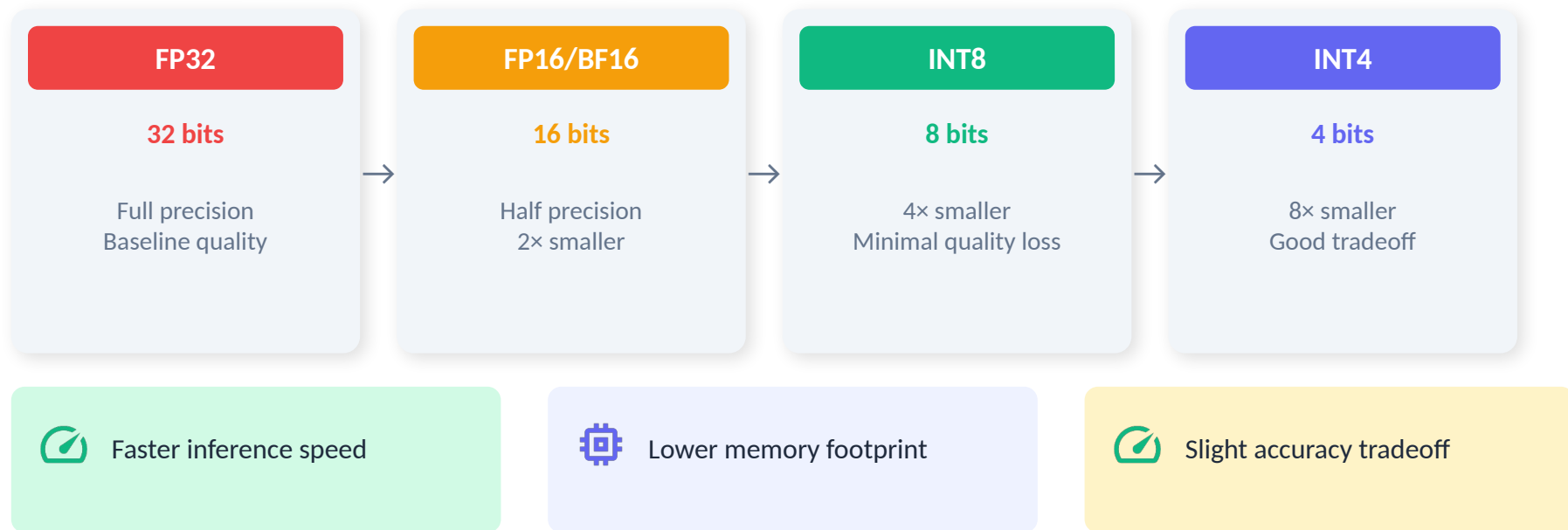
Memory Requirements (GB)





What is Quantization?

Quantization reduces the numerical precision of model weights and activations — for example, from 32-bit floats (FP32) down to 8-bit integers (INT8) or 4-bit integers (INT4). This directly cuts memory usage and speeds up computation.



Types of Quantization

Post-Training Quantization

(PTQ)

- Applied after training is complete
- No need to retrain or fine-tune
- Fast and simple to apply
- Small accuracy drop at INT4
- Popular: GPTQ, AWQ, bitsandbytes

VS

Quantization-Aware Training

(QAT)

- Simulates quantization during training
- Model adapts to lower precision
- Better accuracy than PTQ
- Requires training pipeline access
- Examples: QLoRA, LLM.int8()



PRUNING

What is Pruning?

Pruning removes redundant or low-importance weights from a trained neural network, creating a sparser model that requires fewer parameters and fewer floating-point operations (FLOPs).

~90%

of weights can be
zeroed out with
minimal quality loss

2 – 10×

speedup achievable
with sparse-aware
hardware support

SparseGPT

prunes LLMs to 50%
sparsity in one pass
<1% perplexity loss



Types of Pruning

Unstructured Pruning

Individual weight removal

- Removes individual weights (not structures)
- Highest accuracy at high sparsity
- Creates sparse, irregular weight matrices
- Needs sparse kernels for real speedups
- Key tools: SparseGPT, Wanda

VS

Structured Pruning

Whole structure removal

- Removes neurons, heads, or entire layers
- Dense matrices — no sparse hardware needed
- Compatible with standard GPU inference
- Higher accuracy cost than unstructured
- Key tools: LLM-Pruner, ShortGPT



Quantization vs Pruning: At a Glance

Aspect	Quantization	Pruning
What changes	Precision of weights & activations (bit-width)	Number of weights (zeros or removed structures)
Key technique	Map FP32 → INT8 / INT4	Zero out or remove low-magnitude weights
Memory savings	2× – 8× (FP16 → INT4)	Up to 10× at high sparsity
Speed gains	2× – 4× with INT8/INT4 kernels	2× – 10× with sparse-capable hardware
Accuracy impact	Low at INT8 (≈ FP16); moderate at INT4	Low ≤ 50% sparsity; higher beyond that
Hardware needed	Standard GPU/CPU with INT kernels	Sparse HW for unstructured (e.g., NVIDIA A100 2:4)
Ease of use	Very easy — PTQ needs no retraining	Moderate — often needs fine-tuning to recover quality
Popular tools	bitsandbytes, GPTQ, AWQ, llama.cpp	SparseGPT, Wanda, LLM-Pruner, ShortGPT



Real-World Tools & Frameworks

Quantization

llama.cpp

CPU-friendly GGUF format. Runs Mistral, LLaMA, Phi on consumer hardware at INT4/INT8 with no GPU needed.

Quantization

bitsandbytes

8-bit & 4-bit quantization for PyTorch. Drop-in HuggingFace integration. Powers QLoRA fine-tuning.

Quantization

GPTQ

Layer-wise second-order quantization. State-of-the-art accuracy at INT4. Wide model support.

Pruning

SparseGPT

One-shot unstructured pruning using approximate sparse regression. Achieves 50% sparsity with no retraining.

Pruning

Wanda

Simple, fast pruning using weight magnitude $\times$ activation norm. No back-propagation needed. Very fast.

Pruning

LLM-Pruner

Structured pruning that removes non-critical model sub-structures followed by fast LoRA-based recovery.



Key Takeaways

Quantization is the easiest win

- PTQ (GPTQ, AWQ, bitsandbytes) requires no retraining and delivers 4–8× memory savings with minimal quality loss.

Pruning unlocks model sparsity

- Unstructured pruning (SparseGPT, Wanda) can remove 50%+ of weights. Structured pruning creates smaller, dense sub-models.

Combine for maximum efficiency

- QLoRA = Quantization + LoRA fine-tuning. GPTQ + sparse kernels can deliver exceptional throughput at scale.

Hardware alignment is critical

- INT8/INT4 gains require optimized kernels (vLLM, TensorRT). Unstructured pruning needs sparse-capable hardware for real speedups.